

# STREAK

## Production-Ready Product & Engineering Requirements

**Challenge:** Dategain Developer Challenge — Part 2

**Product:** Streak

**Version:** V1

**Objective:** Build a small, production-quality daily guessing game that is reliable enough to ship, not merely a visual demo.

---

## 0. THE PRODUCT IN ONE SENTENCE

**Streak gives every player one puzzle and one guess per day; a correct answer extends the player's streak, while an incorrect answer or missed day resets it.**

The product must feel:

- fast
- simple
- premium
- fair
- reliable
- addictive without being manipulative
- understandable without instructions

The product must **not** feel like a hackathon demo.

---

## 1. NON-NEGOTIABLE PRODUCT PRINCIPLES

These rules override implementation convenience.

### 1.1 One guess means ONE guess

A player gets exactly one attempt for a puzzle/day.

This is enforced:

- in the UI
- in the API
- in the database

Never rely on frontend state for this rule.

---

## **1.2 The server owns the truth**

The client must never determine:

- today's puzzle
- whether the answer is correct
- current streak
- whether the player has already played
- whether a day was missed
- what the current game date is

The server/database owns all game state.

---

## **1.3 The answer is secret**

The correct answer must NEVER be sent to the frontend before the player submits.

Do not put it in:

- HTML
- JavaScript bundles
- API responses
- hidden DOM elements
- frontend state
- source maps
- public configuration

The answer is returned only after the player's attempt has been permanently recorded.

---

## **1.4 A network failure must not consume a guess**

If the server does not successfully record the attempt, the user has not played.

The frontend must not mark the game as complete based on optimistic UI.

---

## **1.5 A duplicate request must not create a second attempt**

Double-clicks, retries, browser refreshes, network retries, or concurrent requests must never allow two attempts.

Database constraints are mandatory.

---

## 1.6 Streak calculations must be deterministic

The same database state must always produce the same streak.

Do not rely on frontend date calculations.

---

## 1.7 No unnecessary features

Do not build features simply to make the application look bigger.

The core experience is:

**Puzzle → Guess → Result → Streak → Return tomorrow**

Everything else is secondary.

---

# 2. PRODUCT OBJECTIVE

The objective is not simply to make users solve puzzles.

The objective is to create a **daily ritual**.

A successful user experience should be:

"I have one puzzle today. I have one chance. I want to keep my streak alive. I'll come back tomorrow."

The product should take approximately:

**30–60 seconds per daily session.**

It should be possible to understand the game without reading a tutorial.

---

# 3. PRIMARY USER

A casual player who:

- enjoys puzzles
- likes short daily games
- enjoys maintaining streaks

- wants a quick challenge
  - may share achievements
  - does not want complicated onboarding
- 

## 4. CORE USER JOURNEY

### First visit

```
Landing
↓
Choose display name
↓
Player created
↓
Today's puzzle
↓
Submit one guess
↓
Result
↓
Streak
↓
Return tomorrow
```

### Returning user

```
Open app
↓
Identify existing player
↓
Determine today's game state
↓
If not played → show today's puzzle
↓
If already played → show completed state
```

---

## 5. GAME RULES

### 5.1 Daily puzzle

There is exactly one official puzzle for each game day.

For V1:

one game day = one calendar day

The canonical game timezone is:

**Asia/Kolkata**

The server determines the current game date.

The browser's local timezone must never determine the game date.

---

## 6. PUZZLE TYPE

V1 uses:

### Mystery Word / Riddle

Example:

I have cities but no houses, forests but no trees, and water but no fish.

Input:

MAP

The answer is stored server-side.

---

## 7. PUZZLE ENGINE

The backend must use a generic puzzle model rather than hardcoding the application around one puzzle.

Supported model:

```
WORD
RIDDLE
NUMBER
YES_NO
TRIVIA
IMAGE
```

Only `WORD` /riddle needs to be implemented in V1.

The architecture must allow future puzzle types without rewriting:

- player logic
- streak logic
- attempt logic
- daily scheduling

---

## 8. PUZZLE DATA

Each puzzle must contain:

```
id
game_date
type
title
question
answer
difficulty
status
created_at
updated_at
```

Example:

```
{
  "type": "WORD",
  "title": "Today's Mystery",
  "question": "I have cities but no houses...",
  "difficulty": "MEDIUM"
}
```

The API response must deliberately omit `answer` .

---

## 9. PUZZLE CONTENT POLICY

Every published puzzle must:

- have one intended answer
- be understandable
- avoid unnecessary ambiguity
- avoid offensive content
- avoid discriminatory content
- avoid culturally dependent assumptions where possible
- have been reviewed before publication
- have a known difficulty
- have a valid game date

Do not generate puzzles dynamically with an LLM during gameplay.

For V1, curated puzzles are more reliable and cheaper.

---

## 10. CONTENT PIPELINE

Production architecture should support:

```
Draft
↓
Review
↓
Scheduled
↓
Published
↓
Archived
```

A puzzle cannot accidentally become today's puzzle simply because it exists in the database.

Only:

```
status = PUBLISHED  
AND  
game_date = current_game_date
```

can become active.

There must be at most one published puzzle per game date.

Database constraint:

```
UNIQUE(game_date)
```

for the relevant published/scheduled puzzle model.

---

## 11. PLAYER IDENTITY

Authentication is intentionally out of scope.

The challenge explicitly allows:

username or device ID.

Use an anonymous UUID-based player identity.

First visit:

```
What's your name?  
  
[ Enter name ]  
  
[ Start Playing ]
```

Backend creates:

```
player_id = UUID
```

The browser stores the player ID.

The display name is cosmetic.



The player ID is the actual identity.

---

## 12. PLAYER ID SECURITY

Never allow the client to submit arbitrary streak values.

The client may send:

```
player_id  
guess
```

The server determines everything else.

Do not accept:

```
current_streak  
last_played  
is_correct  
game_date
```

from the client.

---

## 13. DATABASE

Use:

**PostgreSQL**

Recommended implementation:

- PostgreSQL
  - Prisma ORM
  - migrations committed to Git
-

## 14. DATABASE SCHEMA

### players

```
id UUID PRIMARY KEY
display_name VARCHAR(30) NOT NULL
current_streak INTEGER NOT NULL DEFAULT 0
longest_streak INTEGER NOT NULL DEFAULT 0
total_games INTEGER NOT NULL DEFAULT 0
total_correct INTEGER NOT NULL DEFAULT 0
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

### puzzles

```
id UUID PRIMARY KEY
game_date DATE NOT NULL
type VARCHAR NOT NULL
title VARCHAR NOT NULL
question TEXT NOT NULL
answer TEXT NOT NULL
difficulty VARCHAR NOT NULL
status VARCHAR NOT NULL
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL
```

Constraint:

```
UNIQUE(game_date)
```

for the active puzzle schedule.

### attempts

```
id UUID PRIMARY KEY
player_id UUID NOT NULL
puzzle_id UUID NOT NULL
guess TEXT NOT NULL
```

```
is_correct BOOLEAN NOT NULL
submitted_at TIMESTAMP NOT NULL
```

Critical constraint:

```
UNIQUE(player_id, puzzle_id)
```

This is mandatory.

---

## 15. DATABASE INTEGRITY

Foreign keys must be enforced:

```
attempt.player_id → player.id
attempt.puzzle_id → puzzle.id
```

Do not allow orphan attempts.

Use database transactions for:

```
attempt creation
+
player streak update
```

These operations must succeed or fail together.

---

## 16. STREAK RULES

Let:

```
today = server's current game date
lastPlayed = player's last played game date
```

### New player

Correct answer:

streak = 1

Incorrect answer:

streak = 0

---

### **Played yesterday + correct today**

streak = previous streak + 1

---

### **Played yesterday + incorrect today**

streak = 0

---

### **Missed one or more days**

If the player returns after missing a day:

Correct answer:

streak = 1

Incorrect answer:

streak = 0

---

### **Already played today**

Reject the attempt.

The streak must not change.

---

## 17. LONGEST STREAK

`longest_streak` should only increase.

Example:

```
current = 9
longest = 14
```

Correct answer:

```
current = 10
longest = 14
```

If current reaches 15:

```
current = 15
longest = 15
```

Resetting current streak must never erase longest streak.

---

## 18. STREAK STATE MACHINE

The backend should conceptually operate using:

```
NEW
↓
PLAYED_CORRECT
↓
ACTIVE_STREAK

ACTIVE_STREAK
↓ correct
ACTIVE_STREAK + 1

ACTIVE_STREAK
↓ wrong
BROKEN

ACTIVE_STREAK
```

```
↓ missed day  
BROKEN  
  
BROKEN  
↓ correct  
NEW STREAK = 1
```

This should be covered by automated tests.

---

## 19. ONE-GUESS ENFORCEMENT

When:

```
POST /api/v1/game/guess
```

the backend must:

1. validate player
2. determine current game date
3. load today's puzzle
4. check existing attempt
5. reject if one exists
6. normalize guess
7. compare answer
8. create attempt
9. calculate streak
10. update player
11. return result

All state-changing operations should happen in a database transaction.

---

## 20. CONCURRENT REQUESTS

The application must handle:

```
Request A → submit guess  
Request B → submit guess
```

arriving simultaneously.

The unique constraint:

```
UNIQUE(player_id, puzzle_id)
```

must prevent duplicate attempts.

The second request should return a controlled response:

```
ALREADY_PLAYED
```

No duplicate streak update.

---

## 21. ANSWER NORMALIZATION

Before comparison:

```
trim whitespace  
convert to lowercase  
collapse repeated whitespace
```

Therefore:

```
" MAP "  
"map"  
"Map"  
"MAP"
```

are equivalent.

Do not use fuzzy matching in V1.

Do not use AI to decide correctness.

If a puzzle legitimately requires multiple accepted answers, those alternatives must be explicitly stored server-side.

---

## 22. API CONTRACT

### Create player

```
POST /api/v1/players
```

Request:

```
{
  "displayName": "Shoaib"
}
```

Response:

```
{
  "playerId": "uuid",
  "displayName": "Shoaib"
}
```

### Get today's game

```
GET /api/v1/game/today?playerId=uuid
```

Response:

```
{
  "gameDate": "2026-08-13",
  "puzzle": {
    "id": "uuid",
    "type": "WORD",
    "title": "Today's Mystery",
    "question": "...",
    "difficulty": "MEDIUM"
  },
  "player": {
    "currentStreak": 12,
    "longestStreak": 18,
    "hasPlayedToday": false
  }
}
```



```
}  
}
```

Never return the answer.

---

## Submit guess

```
POST /api/v1/game/guess
```

Request:

```
{  
  "playerId": "uuid",  
  "guess": "map"  
}
```

Success:

```
{  
  "correct": true,  
  "previousStreak": 12,  
  "currentStreak": 13,  
  "longestStreak": 18,  
  "answer": "map"  
}
```

The answer is returned ONLY after successful attempt recording.

---

## Player statistics

```
GET /api/v1/players/:playerId/stats
```

Returns:

```
current streak  
longest streak  
total games
```

```
total correct
accuracy
```

---

## 23. API ERROR CONTRACT

Every API error should use a consistent shape.

Example:

```
{
  "error": {
    "code": "ALREADY_PLAYED",
    "message": "You've already used today's guess."
  }
}
```

Potential error codes:

```
INVALID_REQUEST
PLAYER_NOT_FOUND
PUZZLE_NOT_FOUND
ALREADY_PLAYED
RATE_LIMITED
INTERNAL_ERROR
```

Do not expose stack traces to clients.

---

## 24. IDEMPOTENCY

Repeated requests caused by:

- double clicks
- browser retries
- network retries

must not create duplicate attempts.

Database uniqueness is the final protection.

The frontend should additionally disable submission while a request is in progress.

---

## 25. FRONTEND ARCHITECTURE

Recommended:

```
React
TypeScript
Vite
Tailwind CSS
React Query / TanStack Query
Zod
```

Use a clean component structure.

Example:

```
src/
  components/
    GameCard
    GuessInput
    StreakDisplay
    ResultCard
    Header
    LoadingState
    ErrorState

  pages/
    Home

  hooks/
    usePlayer
    useTodayGame
    useSubmitGuess

  services/
    api

  lib/
    validation
    date

  types/
```

```
game
player
```

Do not put business logic directly inside UI components.

---

## 26. UI STATES

The frontend must explicitly support:

```
FIRST_VISIT
LOADING
READY_TO_PLAY
SUBMITTING
CORRECT
INCORRECT
ALREADY_PLAYED
NETWORK_ERROR
SERVER_ERROR
INVALID_INPUT
```

No blank screens.

No undefined states.

---

## 27. FIRST VISIT UI

Minimal onboarding:

```
STREAK

One puzzle.
One guess.
Every day.

[ Your name ]

[ START ]
```

Rules can be shown in one short sentence:

Get it right to extend your streak. Miss or get it wrong and it resets.

No multi-screen onboarding.

---

## 28. GAME UI

The puzzle must dominate the screen.

Structure:

STREAK

 12

TODAY'S PUZZLE

[ Question ]

Your one guess

[ Input ]

[ SUBMIT GUESS ]

One guess. Make it count.

---

## 29. SUBMISSION UX

Before submission:

SUBMIT GUESS

During:

CHECKING...

After server response:

show result.

Prevent multiple clicks.

---

## 30. CORRECT STATE

Example:

CORRECT 🎉

🔥 13 DAY STREAK

You extended your streak.

12 → 13

Longest streak  
18 days

Come back tomorrow.

The answer may be displayed now.

---

## 31. INCORRECT STATE

Example:

NOT THIS TIME

Your answer:  
OCEAN

Correct answer:  
MAP

Your streak has reset.

Tomorrow is a new chance.

Do not use humiliating language.

---

## 32. COMPLETED-TODAY STATE

If user returns after playing:

TODAY COMPLETE ✓

🔥 13 DAY STREAK

You've already made today's guess.

Next puzzle in:

05:42:18

Come back tomorrow.

No second attempt UI.

---

## 33. MISSED-DAY STATE

When appropriate:

Your previous streak ended  
because you missed a day.

Start a new streak today.

Then show today's puzzle.

Do not block the user from starting again.

---

## 34. STREAK PRESENTATION

Current streak:

**highest visual priority after the puzzle.**

Longest streak:

secondary.

Statistics:

tertiary.

Avoid dashboard clutter.

---

## 35. PERSONAL STATS

Include:

Current streak  
Longest streak  
Games played  
Correct answers  
Accuracy

Keep this secondary to the core daily game.

---

## 36. DAILY MOTIVATION

The product should create anticipation without artificial manipulation.

Useful:

Your streak is waiting tomorrow.

Useful:

3 more days to beat your longest streak.

Avoid:

- fake scarcity
  - fake countdowns
  - aggressive notifications
  - dark patterns
  - forced sharing
-



## 37. COUNTDOWN

After completion:

```
Next puzzle in  
HH:MM:SS
```

The countdown is visual only.

The backend determines when the next puzzle actually becomes available.

Client time must never grant access early.

---

## 38. MIDNIGHT / GAME-DAY TRANSITION

The client should periodically refresh the game state.

After game-day rollover:

```
GET /game/today
```

The new puzzle should appear.

The backend is authoritative.

---

## 39. REFRESH BEHAVIOR

Before playing:

Refresh → today's puzzle remains.

After playing:

Refresh → completed state remains.

Close browser:

Reopen → player identity remains.

Restart backend:

Data remains.

Restart database:

Data remains if persistence is configured correctly.

---

## 40. NETWORK FAILURE

If submitting:

CHECKING...

and request fails:

Couldn't submit your guess.

Your attempt was not recorded.

[ TRY AGAIN ]

Never silently consume the attempt.

---

## 41. ACCESSIBILITY

Must include:

- semantic HTML
- proper form labels
- keyboard navigation
- focus states
- screen-reader labels
- accessible error messages
- sufficient contrast
- reduced-motion support
- no information communicated only by color

The game must be usable without a mouse.

---

## 42. RESPONSIVE DESIGN

Mobile-first.

Must support:

- 320px+
- tablet
- desktop

The core game should not become a completely different interface on desktop.

---

## 43. VISUAL DESIGN

Design direction:

**Premium + minimal + playful.**

Avoid:

- excessive gradients
- excessive glassmorphism
- excessive animation
- crowded dashboards
- unnecessary cards
- template-looking UI

Typography and spacing should carry the design.

The puzzle should visually dominate.

---

## 44. MOTION

Only use motion for:

- page/state transition
- correct result
- streak increment
- subtle countdown transition

Respect:

prefers-reduced-motion

No animation should block interaction.

---

## 45. SECURITY

Even though V1 has no authentication, implement:

- input validation
  - maximum input lengths
  - rate limiting
  - CORS restrictions
  - HTTPS deployment
  - secure headers
  - environment variables
  - no secrets in source
  - no stack traces in production
  - server-side game logic
  - database constraints
- 

## 46. INPUT VALIDATION

Display name:

1-30 characters

Guess:

1-100 characters

Reject:

- empty values
- oversized payloads
- malformed IDs

Validate with Zod or equivalent.

---

## 47. RATE LIMITING

Rate-limit:

```
player creation  
game retrieval  
guess submission
```

Guess submission should have a stricter limit.

The purpose is not sophisticated anti-cheat.

The purpose is preventing:

- accidental abuse
- brute force
- automated spam
- API flooding

---

## 48. CHEATING PREVENTION

Never trust:

```
client date  
client streak  
client correctness  
client puzzle ID
```

The server determines the puzzle based on the current game date.

The server determines the answer.

The server determines streak.

The database enforces one attempt.

---

## 49. DEVICE / ID ABUSE

V1 does not attempt to perfectly prevent users from creating multiple anonymous identities.

That is a conscious tradeoff.

The system should prevent cheap abuse but not spend engineering time building full authentication.

Future:

```
anonymous player
↓
optional account upgrade
↓
Google/email authentication
↓
identity merge
```

---

## 50. PERFORMANCE

Targets:

### Frontend

Fast initial render.

Avoid unnecessary dependencies.

Lazy-load non-critical code.

### API

Normal requests target:

```
<300ms
```

under normal conditions.

### Guess

Target:

```
<500ms
```

under normal conditions.

There is no reason for an AI request in the critical path.

---

## 51. CACHING

Today's puzzle content can be cached carefully.

But player-specific state must remain fresh.

Never cache:

```
current streak
hasPlayedToday
```

in a way that can cause stale game state.

---

## 52. DATABASE TRANSACTIONS

Guess submission must be transactional:

```
BEGIN

check duplicate
↓
evaluate answer
↓
insert attempt
↓
calculate streak
↓
update player

COMMIT
```

Any failure:

```
ROLLBACK
```

The user must not lose a guess because only half of the operation succeeded.

---

## 53. OBSERVABILITY

Backend must include:

### Health check

```
GET /health
```

Response:

```
{  
  "status": "ok"  
}
```

Optionally verify database connectivity.

---

## 54. LOGGING

Use structured logs.

Log:

```
request ID  
endpoint  
status  
duration  
error code
```

Do NOT log:

- database passwords
  - environment secrets
  - unnecessary personal data
  - full user guesses if not required
- 

## 55. ERROR MONITORING

Production should be structured so an error-monitoring service can be added.



For the challenge, console/structured logs are acceptable if external monitoring isn't available on the free tier.

---

## 56. ANALYTICS

Track product behavior.

Events:

```
PLAYER_CREATED  
PUZZLE_VIEWED  
GUESS_SUBMITTED  
GUESS_CORRECT  
GUESS_WRONG  
STREAK_EXTENDED  
STREAK_BROKEN  
GAME_COMPLETED  
SHARE_CLICKED
```

Avoid collecting unnecessary personal information.

---

## 57. KEY PRODUCT METRICS

Primary:

**Daily completed players**

Secondary:

**D1 retention**

Percentage who return the next day.

**D7 retention**

Percentage who return seven days later.

**Puzzle completion rate**

Players who view today's puzzle and submit.

**Average streak**

**Longest streak distribution**

**Correct-answer rate**

**Streak break rate**

---

## 58. PRODUCT LEARNING

Puzzle-level analytics should eventually identify:

- too easy
- too difficult
- ambiguous
- unusually high abandonment
- unusually high wrong-answer rate

Example:

Puzzle A  
solve rate: 94%

Probably too easy.

Puzzle B  
solve rate: 4%

Potentially too difficult or unclear.

---

## 59. SHARING

Optional but valuable.

After meaningful milestones:

7 day streak  
14 day streak  
30 day streak

Allow a share card.

Example:

 30 DAY STREAK

30 puzzles.  
30 days.  
Still going.

Play Streak.

Do not expose the current answer.

Do not force users to share.

---

## 60. LEADERBOARD

Not required for the first playable version.

If implemented, it should rank based on meaningful skill/consistency, not simply login time.

Potential ranking:

1. current streak
2. longest streak
3. total correct

Do not let users buy leaderboard position.

---

## 61. NOTIFICATIONS

Future feature.

Opt-in only.

Example:

Your 12-day streak is waiting.

Never send repeated notifications simply to increase engagement.

---

## 62. MONETIZATION

Do not compromise the core rule.

Never sell:

"Another guess."

The value of Streak comes from the one-guess rule.

Potential future monetization:

- premium statistics
- custom challenges
- premium puzzle categories
- cosmetic personalization
- limited streak protection

Not required for V1.

---

## 63. ADMIN / OPERATIONS

A production system needs a way to manage content.

Minimum operational capability:

- create puzzle
- edit puzzle
- schedule puzzle
- publish puzzle
- unpublish puzzle
- inspect puzzle status

A full admin frontend is optional for the challenge.

Database-backed content management is sufficient for V1.

---

## 64. FAILURE: NO PUZZLE FOR TODAY

This is a critical operational case.

If no puzzle exists for today's date:

The user must NOT see a broken page.

Show:

Today's puzzle isn't available yet. We're fixing it.

Log an operational error.

Do not silently select tomorrow's puzzle.

Do not expose the answer.

---

## 65. FAILURE: DATABASE UNAVAILABLE

The application should show:

We're having trouble loading today's challenge. Please try again.

Do not show misleading streak data.

Do not allow guessing if the server cannot reliably persist the attempt.

---

## 66. FAILURE: SERVER RESTART

No game state should be lost.

Everything important lives in PostgreSQL.

Never use in-memory variables for:

- streaks
  - attempts
  - today's puzzle
  - player identity
-

## 67. TESTING REQUIREMENTS

### Unit tests

Mandatory:

- answer normalization
  - answer comparison
  - first-game streak
  - consecutive-day streak
  - wrong answer reset
  - missed-day reset
  - longest streak
  - duplicate detection
  - game-date calculation
- 

### Integration tests

Mandatory:

```
create player
↓
get today's puzzle
↓
submit correct guess
↓
verify attempt
↓
verify streak
```

Also:

```
submit twice
```

must fail.

---

## API tests

Test:

- valid request
  - invalid request
  - missing player
  - malformed UUID
  - oversized guess
  - duplicate guess
  - rate limiting
  - missing puzzle
- 

## 68. CRITICAL EDGE CASE TEST MATRIX

| Scenario                  | Expected        |
|---------------------------|-----------------|
| New player + correct      | Streak 1        |
| New player + wrong        | Streak 0        |
| Correct consecutive day   | +1              |
| Wrong after active streak | 0               |
| Missed day + correct      | 1               |
| Missed day + wrong        | 0               |
| Second attempt            | Rejected        |
| Double-click              | One attempt     |
| Two concurrent requests   | One attempt     |
| Refresh before play       | Same puzzle     |
| Refresh after play        | Completed state |
| Browser reopened          | Player persists |
| Backend restarted         | Data persists   |
| DB restarted              | Data persists   |
| Client changes date       | No effect       |
| Client changes streak     | Impossible      |
| No puzzle today           | Graceful error  |

| Scenario        | Expected           |
|-----------------|--------------------|
| Network failure | Guess not consumed |

---

## 69. DEPLOYMENT

Recommended:

### Frontend

Vercel

### Backend

Render

### Database

Supabase PostgreSQL

Architecture:

```
Browser
  ↓ HTTPS
Vercel
  ↓
React application
  ↓ HTTPS
Render
  ↓
Node.js API
  ↓
Supabase PostgreSQL
```

---

## 70. ENVIRONMENT VARIABLES

Example:

```
DATABASE_URL
API_BASE_URL
```



```
NODE_ENV  
CORS_ORIGIN
```

Never commit:

```
.env  
.env.local
```

Use:

```
.env.example
```

with placeholder values.

---

## 71. CORS

Production backend must allow only the deployed frontend origin.

Development can allow localhost.

Do not use:

```
Access-Control-Allow-Origin: *
```

in production if credentials/security policy does not require it.

---

## 72. README

README must include:

1. Product overview
2. Live link
3. Architecture
4. Tech stack
5. Local setup
6. Environment variables
7. Database setup
8. API overview
9. Game rules

10. Streak logic
  11. Security decisions
  12. Testing
  13. Deployment
  14. Tradeoffs
  15. Future improvements
- 

## 73. GIT WORKFLOW

Commit progressively.

Recommended history:

```
chore: initialize project
feat: add database schema
feat: add puzzle API
feat: add player API
feat: implement streak engine
feat: enforce one guess per day
feat: build daily puzzle UI
feat: add result states
feat: add player statistics
test: cover streak edge cases
fix: handle duplicate submissions
fix: handle game-day rollover
chore: production configuration
docs: update README
```

Do not squash everything into one final commit.

---

## 74. CODE QUALITY

Requirements:

- TypeScript strict mode
- no unnecessary `any`
- centralized API client
- reusable validation
- clear naming
- small functions
- separation of business logic
- no duplicated streak calculations

- no hardcoded secrets
- no dead code
- no commented-out experiments
- no massive components
- no business logic hidden in UI

---

## 75. ARCHITECTURAL SEPARATION

The streak calculation should exist independently from HTTP.

Conceptually:

```
Controller
  ↓
Game Service
  ↓
Streak Service
  ↓
Repository
  ↓
Database
```

This makes the most important business logic testable without needing the entire application running.

---

## 76. DO NOT OVERENGINEER

Do NOT introduce:

- microservices
- Kafka
- Redis unless demonstrably necessary
- Kubernetes
- WebSockets
- GraphQL
- AI agents
- event-driven architecture
- complex authentication

The product does not need them.

A modular monolith is the correct architecture.

---

## 77. WHY MODULAR MONOLITH

At the scale of this product:

```
React
+
Node API
+
PostgreSQL
```

is sufficient.

It is:

- cheap
- easy to deploy
- easy to debug
- easy to maintain
- fast enough
- appropriate for V1

If scale grows, individual services can be extracted later.

---

## 78. AI USAGE

AI is NOT required for the game.

Do not add AI merely because the hiring challenge says AI tools are allowed.

Use AI for:

- development assistance
- code generation
- testing assistance
- documentation assistance

The actual game should remain deterministic.

Potential future AI use:

- puzzle generation
- puzzle quality review
- difficulty prediction

But these belong outside the critical gameplay path.

---

## 79. PRODUCTION READINESS CHECKLIST

Before declaring the app complete:

### Product

- ☐ Daily puzzle works
- ☐ One guess rule works
- ☐ Correct answer extends streak
- ☐ Wrong answer resets streak
- ☐ Missed day resets streak
- ☐ Completed state works
- ☐ Longest streak works

### Backend

- ☐ Server determines game date
- ☐ Server determines answer
- ☐ Server determines streak
- ☐ Input validation
- ☐ Rate limiting
- ☐ Error handling
- ☐ Transactions
- ☐ Duplicate protection
- ☐ Health endpoint

### Database

- ☐ Migrations
- ☐ Foreign keys
- ☐ Unique constraints
- ☐ Persistent storage
- ☐ Seed data
- ☐ No secrets committed

### Frontend

- ☐ Mobile responsive
- ☐ Loading states
- ☐ Error states
- ☐ Empty states
- ☐ Accessible forms
- ☐ Double-click protection
- ☐ Refresh behavior

- ☐ Completed state

## Testing

- ☐ Unit tests
- ☐ Integration tests
- ☐ Edge cases
- ☐ Manual mobile test
- ☐ Production smoke test

## Deployment

- ☐ Frontend live
- ☐ Backend live
- ☐ Database live
- ☐ Environment variables configured
- ☐ CORS configured
- ☐ HTTPS
- ☐ Health endpoint working

## Documentation

- ☐ README
- ☐ Architecture diagram
- ☐ Setup instructions
- ☐ API documentation
- ☐ Tradeoffs documented

---

# 80. DEFINITION OF DONE

The product is NOT done when:

"I can play it once."

The product is done when:

1. A new user can discover the game without explanation.
2. A user can play today's puzzle.
3. The server validates the guess.
4. The server persists the attempt.
5. The streak is calculated correctly.
6. A second attempt is impossible.
7. Closing/reopening the browser preserves the player.
8. Refreshing after submission preserves the result.
9. Wrong answers reset correctly.
10. Missed days reset correctly.

11. Database restarts do not lose data.
  12. Network failures do not consume guesses.
  13. Concurrent requests cannot bypass the one-guess rule.
  14. The application behaves correctly on mobile.
  15. Errors are understandable.
  16. The backend is secured reasonably for a public V1.
  17. The application is deployed.
  18. Tests cover critical business logic.
  19. The README explains important decisions.
  20. Another developer can clone the repository and understand how the system works.
- 

## 81. CUT ORDER IF TIME BECOMES LIMITED

If implementation time becomes constrained, features are removed in this exact order:

### Remove first

1. leaderboard
2. sharing
3. advanced statistics
4. animations
5. countdown
6. admin UI
7. analytics dashboard

### NEVER remove

1. server-side validation
  2. database persistence
  3. streak correctness
  4. one-guess enforcement
  5. database uniqueness
  6. transactions
  7. input validation
  8. error handling
  9. responsive UI
  10. testing of core game logic
  11. deployment reliability
- 

## 82. WHAT THE REVIEWER SHOULD EXPERIENCE

A reviewer opens the URL.

Within seconds they understand:

One puzzle. One guess. Keep your streak.

They enter a name.

They solve the puzzle.

They submit.

The result appears.

They refresh.

Their state is still correct.

They try to submit again.

The server refuses.

They close the browser.

They reopen it.

Their streak remains.

They inspect the repository.

They find:

- sensible architecture
- meaningful commits
- clean database schema
- tested business logic
- server-side validation
- clear README
- production deployment

The reviewer should come away thinking:

**"This person understands that a product is more than the UI."**

---



## 83. MASTER IMPLEMENTATION RULE FOR THE AI CODING AGENT

The coding agent must treat this document as the source of truth.

Before implementation:

1. Read the entire PRD.
2. Identify all functional requirements.
3. Identify all non-functional requirements.
4. Produce an implementation plan.
5. Produce the proposed architecture.
6. Produce the database schema.
7. Produce API contracts.
8. Identify ambiguities.
9. Do NOT invent new product behavior.
10. Do NOT add features outside the defined scope.
11. Do NOT begin coding until the plan is internally consistent.

During implementation:

- follow the requirements exactly
- use TypeScript
- keep the architecture modular
- write tests alongside critical business logic
- validate all external input
- never move business rules into the frontend
- never expose the answer before submission
- never trust client-provided game state
- use transactions for guess + streak updates
- enforce uniqueness at the database level
- handle failure states explicitly
- keep the application deployable at every major milestone

After implementation:

1. Run tests.
2. Run lint/type checks.
3. Build frontend.
4. Build backend.
5. Run database migrations from a clean state.
6. Test all critical edge cases.
7. Test production URLs.
8. Test mobile.
9. Inspect network requests.
10. Verify the answer is never exposed before submission.

11. Verify duplicate requests cannot create duplicate attempts.
12. Verify streak calculations across consecutive and missed days.
13. Review the repository for secrets.
14. Update README.
15. Only then consider the application complete.

---

## 84. FINAL PRODUCT PHILOSOPHY

Do not confuse **small scope** with **low quality**.

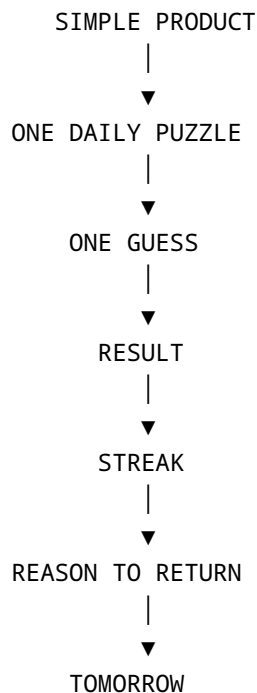
Streak should have:

**A tiny product surface.**

But underneath:

**serious engineering.**

The ideal result is:



The engineering underneath protects that simple experience:

Server Authority  
+  
Database Integrity  
+  
Transactional Updates  
+  
Deterministic Logic  
+  
Security  
+  
Testing  
+  
Observability  
+  
Reliable Deployment

**The product should be simple enough that a user understands it immediately, but robust enough that a developer would be comfortable shipping it.**